Types of RNN

Contents

1. Overview 2
2. Prerequisites 3
3. Many-to-one 3
4. One-to-many 4
5. Many-to-many 5
    5.1 Same length 6
    5.2 Different lengths 7
6. One-to-one 8
7. The types in Keras 9
8. Summary 11
9. Sources 12
10. Key terms 12

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Recurrent neural network (RNN).
- **Leads to:** Sequence-to-sequence (encoder-decoder).

1. Overview

Key point: RNNs are grouped by two questions: is the input a sequence, and is the output a sequence? The answers give many-to-one, one-to-many and many-to-many (same or different lengths). One-to-one, with no sequence on either side, is an ordinary network, not an RNN.

The RNN forward propagation built one RNN: it reads a review word by word and gives one prediction at the end. The same recurrent layer can be wired in other ways, depending on what goes in and what comes out.

Three words recur below. A **sequence** is an ordered list of values, such as the words of a sentence. A **time step** is one position in the sequence, such as one word. The **hidden state** is the vector of numbers the recurrent layer keeps as its memory and passes from one time step to the next. Figures 2 to 6 draw each type **unrolled**: the recurrent layer is drawn once per time step, side by side (see unfolding through time).

Type	Input	Output	Example
Many-to-one	sequence	one value	**sentiment analysis** (G-1769), rating prediction
One-to-many	one value (not a sequence)	sequence	image captioning, music generation
Many-to-many, same length	sequence	sequence, one output per input	part-of-speech tagging, named entity recognition
Many-to-many, different lengths	sequence	sequence of another length	machine translation
One-to-one	one value	one value	image classification (not an RNN)

Figure 1 sorts the types by the two questions of the Key point. Each type also has its own unrolled picture, shown in Figures 2 to 6.

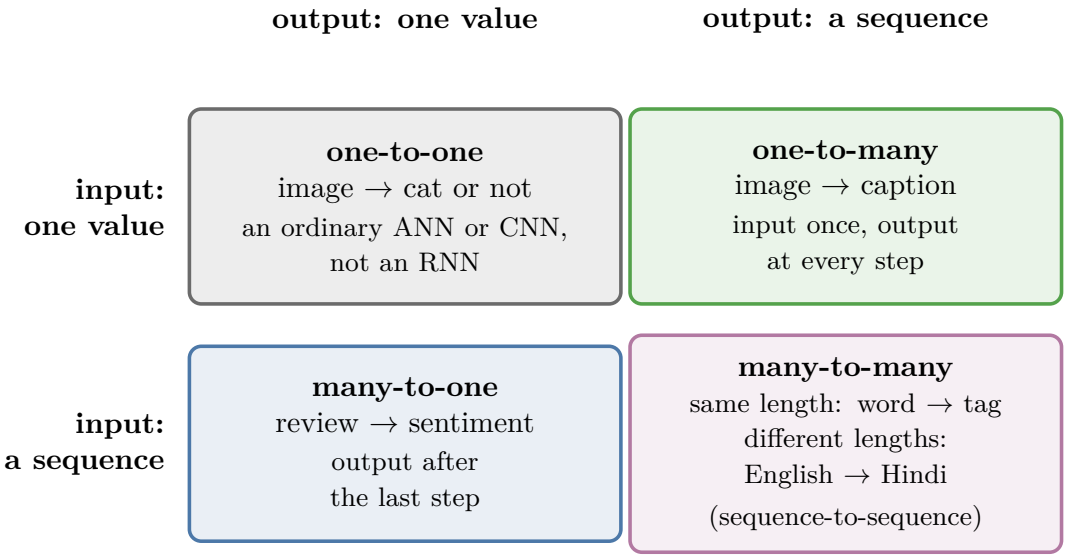


Figure 1: The two questions. Is the input one value or a sequence? Is the output one value or a sequence? Each answer pair gives one type; only the three with a sequence on at least one side are RNNs

The type also changes how backpropagation runs through the network, which the **backpropagation through time** (G-246; **backpropagation through time** (G-246)) covers.

2. Prerequisites

- The why RNNs: sequential and non-sequential data.
- The RNN forward propagation: the recurrent layer, the hidden state h_t and unfolding through time.
- The RNN sentiment analysis: a many-to-one RNN trained in Keras.

3. Many-to-one

> **Key point:** A sequence goes in, one value comes out. The RNN reads every time step and predicts only after the last one.

A **many-to-one** RNN (G-1155) takes a sequence as input, such as a sentence, a string of characters or a time series, and gives a non-sequential output: one number, such as a class label 1 or 0.

- **Sentiment analysis:** the input is a review, read word by word; the output is its sentiment, 1 (positive) or 0 (negative).
- **Rating prediction:** the input is a review; the output is a star rating from 1 to 5. The output is still one value, so this is a **multi-class classification** (G-1266) problem on top of a many-to-one RNN.

many-to-one: sentiment analysis

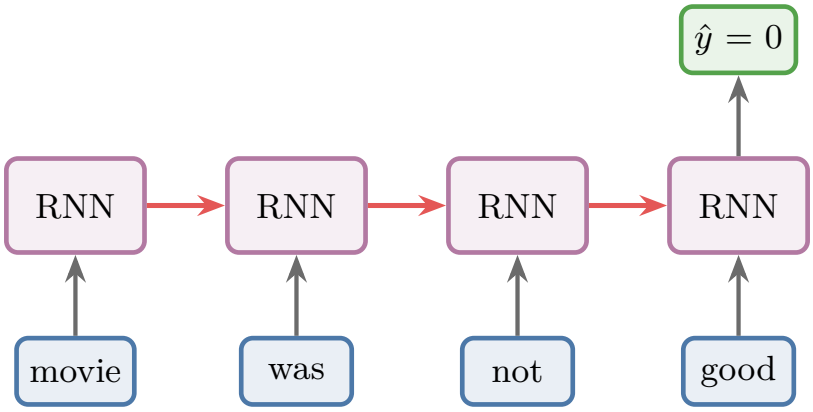


Figure 2: Many-to-one, unrolled: one word enters at each time step, the hidden state passes to the next step (red arrows), and only the last step produces an output

Figure 2 is the RNN of the RNN forward propagation, unrolled: many inputs, one output, hence the name. Goodfellow §10.2 describes this pattern as a network that reads an entire sequence and then produces a single output, a fixed-size summary of the sequence.

> **Python:** In Keras a recurrent layer returns only its last hidden state by default (`return_sequences=False`), which is exactly many-to-one.

```
model = keras.Sequential([
    keras.Input(shape=(4, 5)),      # (time steps, features)
    keras.layers.SimpleRNN(3),      # last hidden state only
    keras.layers.Dense(1, activation="sigmoid")])
# input (3, 4, 5) -> output (3, 1): one number per review
```

4. One-to-many

> **Key point:** One non-sequential input goes in, a sequence comes out. The input enters once, and the RNN produces an output at every time step.

A **one-to-many** RNN (G-1386) takes a non-sequential input and produces a sequence. An image is non-sequential data: it is a grid of numbers with no order in time. The output can be words, a sentence or a time series.

- **Image captioning** (G-918): the input is one image; the output is a sentence that describes it. An image of a man playing cricket might give "a man plays cricket".
- **Music generation:** one input starts the network, and it keeps producing music notes, one per time step.

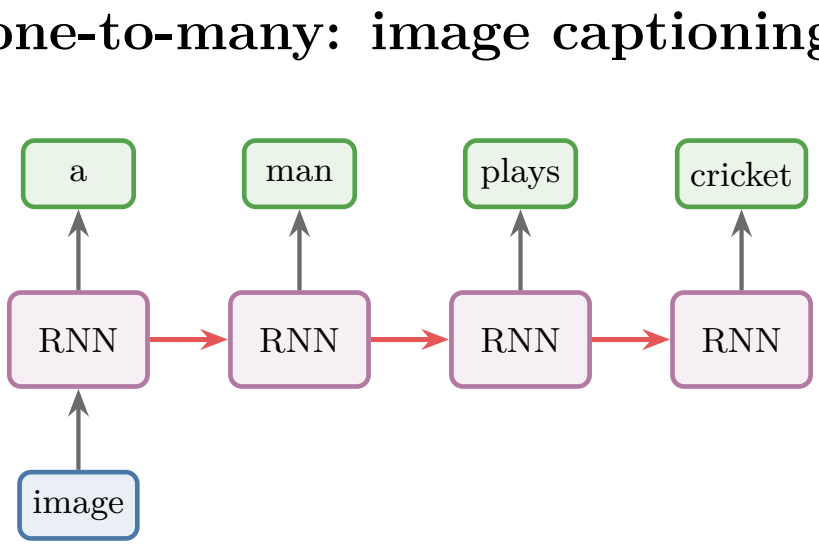


Figure 3: One-to-many, unrolled: the image enters at the first time step only; every time step produces one word of the caption

In Figure 2 the arrows fanned in to one output; in Figure 3 they fan out. The input enters once, at the first time step. At every time step the RNN produces an output and passes its hidden state on to the next step. Goodfellow §10.2.4 names image captioning as the standard task for an RNN that maps one fixed-length vector to a sequence.

> **Extra:** Goodfellow §10.4 gives two ways to feed the single input to such an RNN: as the initial hidden state, or as an input at every time step (and the two can be combined). The Notebook uses the second way: Keras' `RepeatVector(6)` (G-133; copies one vector to several time steps) copies the input to 6 time steps, and the RNN with `return_sequences=True` returns 6 hidden states. An input of shape $(2, 8)$, two "images" of 8 numbers, gives an output of shape $(2, 6, 5)$: 6 words per image, each a probability over 5 words.

5. Many-to-many

> **Key point:** A sequence goes in and a sequence comes out. Such models are also called sequence-to-sequence models. The two sequences can have the same length or different lengths.

A **many-to-many** RNN (G-1154) takes a sequence and returns a sequence, so it is also called a **sequence-to-sequence** (G-1771; seq2seq) model. It comes in two kinds.

5.1 Same length

> **Key point:** One output for every input: the RNN predicts at every time step.

In a **same-length many-to-many** (G-1721) RNN, the output sequence has as many elements as the input sequence.

- **Part-of-speech tagging** (G-1456): for each word of a sentence, predict its part of speech. "my name is Riya" gives pronoun, noun, verb, proper noun: 4 words in, 4 tags out. In **natural language processing** (G-1305; NLP), tagging is a common preprocessing step.
- **Named entity recognition** (G-1300; NER): for each word, decide whether it is an entity, a specific thing the program must act on. In "let's meet at 7 pm at the airport", "7 pm" and "airport" are entities. Chatbots use NER to pick out times and places.

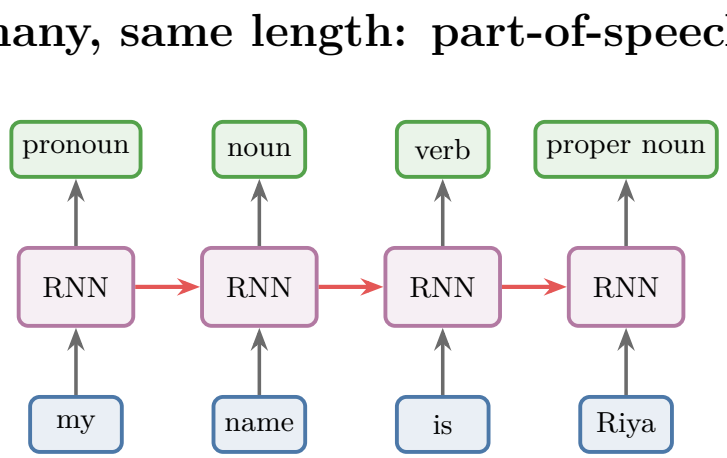


Figure 4: Many-to-many with the same length, unrolled: each time step takes one word and gives one tag

In Figure 4, each time step takes the next input and gives an output, and the hidden state moves on, until the inputs run out. Goodfellow §10.2 lists this as the first RNN pattern: an output at each time step, with recurrent connections between hidden units.

> **Python:** `return_sequences=True` (G-136) makes the recurrent layer return the hidden state of every time step. A `Dense` layer on that 3D output acts on the last axis, so it makes one prediction per time step (Keras documentation, `Dense`).

```
model = keras.Sequential([
    keras.Input(shape=(4, 5)),
    keras.layers.SimpleRNN(3, return_sequences=True),
    keras.layers.Dense(4, activation="softmax")])  # 4 tags
# input (3, 4, 5) -> output (3, 4, 4): 4 tags per word
```

With the same weights, the last of the hidden states returned by `return_sequences=True`

equals the single hidden state returned by `return_sequences=False` (Notebook). The two settings run the same computation and differ only in what they hand on.

5.2 Different lengths

> **Key point:** The output sequence can be longer or shorter than the input. The RNN first reads the whole input (the encoder), and only then starts writing the output (the decoder).

In a **variable-length many-to-many** (G-2072) RNN, the input and output sequences can have different lengths. The standard example is **machine translation** (G-1141): translating a sentence from one language into another. Nothing guarantees that a sentence and its translation have the same number of words: some languages say the same thing in fewer words. "I am going home" has 4 words; its Hindi translation "main ghar ja raha hoon" has 5.

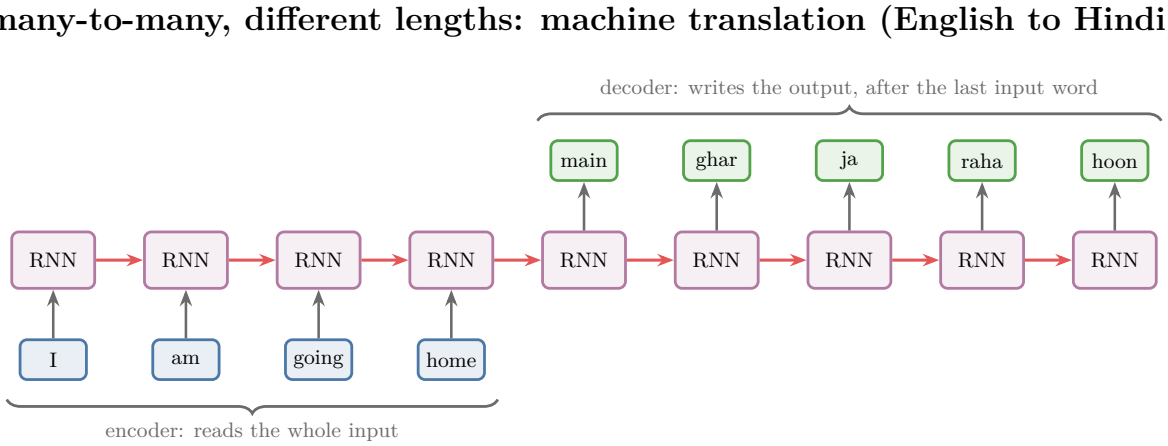


Figure 5: Many-to-many with different lengths, unrolled: the encoder reads all 4 English words; only after the last one does the decoder produce the 5 Hindi words

In Figure 5 the network produces nothing while it reads the input. It starts producing output only after the last input word. The reason is the task: translation does not work word by word. We read the whole sentence, understand its meaning and grammar, and only then decide on the translation. Translating each word as soon as it arrives would lose both the grammar and the context.

The two parts have names.

- The **encoder** (G-682) is the part that reads the input. It emits a summary of the input, usually a simple function of its last hidden state (Goodfellow §10.4).
- The **decoder** (G-564) is the part that writes the output, starting from that summary.

Together they form an **encoder-decoder** architecture. Its advance over the earlier patterns is that the input length and the output length can differ (Goodfellow §10.4). Speech recognition and question answering have the same need (Goodfellow §10.4).

Extra: A minimal encoder-decoder in Keras (Notebook): a `SimpleRNN(3)` encoder keeps only its last hidden state; `RepeatVector(6)` hands it to 6 time steps; a `SimpleRNN(3, return_sequences=True)` decoder and a `Dense(7, activation="softmax")` layer produce 6 words from a 7-word vocabulary. An input of shape $(3, 4, 5)$ gives an output of shape $(3, 6, 7)$: 4 input words become 6 output words. The model is untrained; it only shows the shapes.

6. One-to-one

Key point: Non-sequential in, non-sequential out: no time steps and no recurrence. One-to-one is an ordinary ANN or CNN, so technically it is not an RNN.

A **one-to-one** network (G-1387) takes a non-sequential input and gives a non-sequential output. Image classification is an example: the input is an image, a grid of values, and the output is 1 or 0, cat or not.

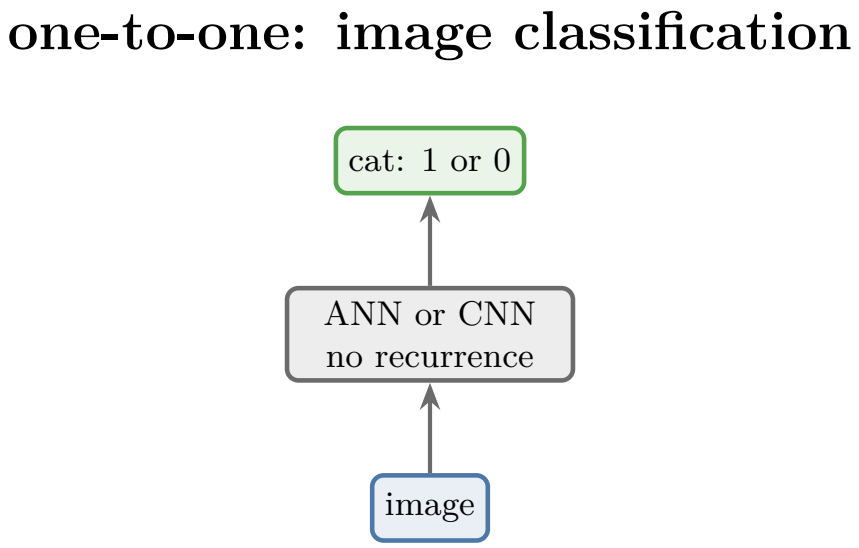


Figure 6: One-to-one: one input, one output, no time steps and no feedback

In Figure 6 everything happens in a single time step, with no feedback connection. Such a network is the plain neural network of the earlier Notes, an ANN or a CNN. One-to-one is listed only to complete the picture; technically there are three types of RNN: many-to-one, one-to-many and many-to-many, the last with its two kinds.

7. The types in Keras

> **Key point:** Two switches build every type: `return_sequences` decides whether the recurrent layer hands on one hidden state or all of them, and `RepeatVector` turns one vector into a sequence.

The Notebook builds one small model of each type, untrained, and checks its shapes. The input is the batch of three one-hot reviews from the RNN forward propagation, shape $(3, 4, 5)$ (3 reviews, 4 words each, 5 numbers per word), or two "images" of 8 numbers, shape $(2, 8)$. Each model ends in a `Dense` layer; with `softmax` it gives one probability per possible tag or word.

Type	Layers	Input shape	Output shape
Many-to-one	`SimpleRNN(3)`, `Dense(1)`	$(3, 4, 5)$	$(3, 1)$
Many-to-many, same length	`SimpleRNN(3, return_sequences=True)`, `Dense(4)`	$(3, 4, 5)$	$(3, 4, 4)$
One-to-many	`RepeatVector(6)`, `SimpleRNN(3, return_sequences=True)`, `Dense(5)`	$(2, 8)$	$(2, 6, 5)$
Many-to-many, different lengths	`SimpleRNN(3)`, `RepeatVector(6)`, `SimpleRNN(3, return_sequences=True)`, `Dense(7)`	$(3, 4, 5)$	$(3, 6, 7)$
One-to-one	`Dense(3)`, `Dense(1)`	$(2, 8)$	$(2, 1)$

Figure 7 follows the shape through each model, layer by layer. Watch the time-step axis: `RepeatVector` creates it, `return_sequences=True` keeps it, and a recurrent layer with `return_sequences=False` removes it.

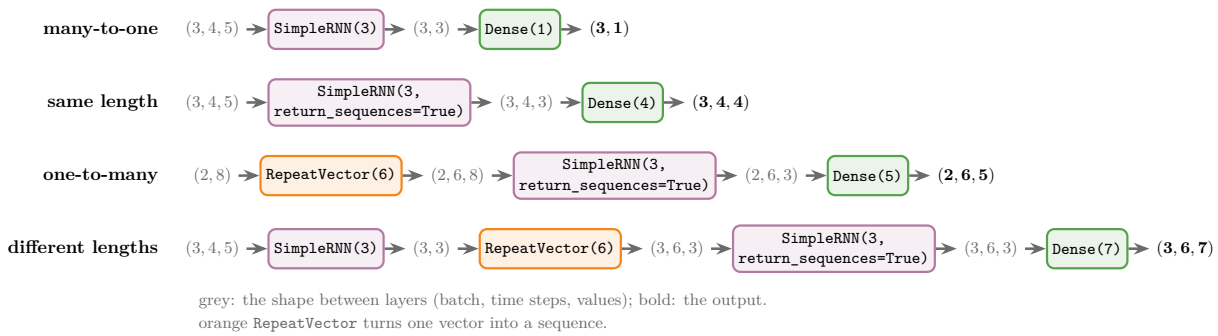


Figure 7: The Notebook's models, layer by layer. Grey: the shape each layer hands on; bold: the model's output. The one-to-one model, two `Dense` layers from $(2, 8)$ to $(2, 1)$, is left out

A 3D output (batch, time steps, values) means one prediction per time step; a 2D output (batch, values) means one prediction per **observation** (G-1374).

8. Summary

Type	Input	Output	Unrolled shape	Example
Many-to-one	sequence	one value	output at the last step only	sentiment analysis
One-to-many	one value	sequence	input at the first step, output at every step	image captioning
Many-to-many, same length	sequence	sequence, same length	input and output at every step	part-of-speech tagging, NER
Many-to-many, different lengths	sequence	sequence, other length	encoder reads all, then decoder writes	machine translation
One-to-one	one value	one value	one step, no recurrence	image classification

Figure 8 runs the four RNN types side by side, one time step per frame. Watch when each row takes an input (blue) and when it gives an output (green): the difference between the types is only this timing.

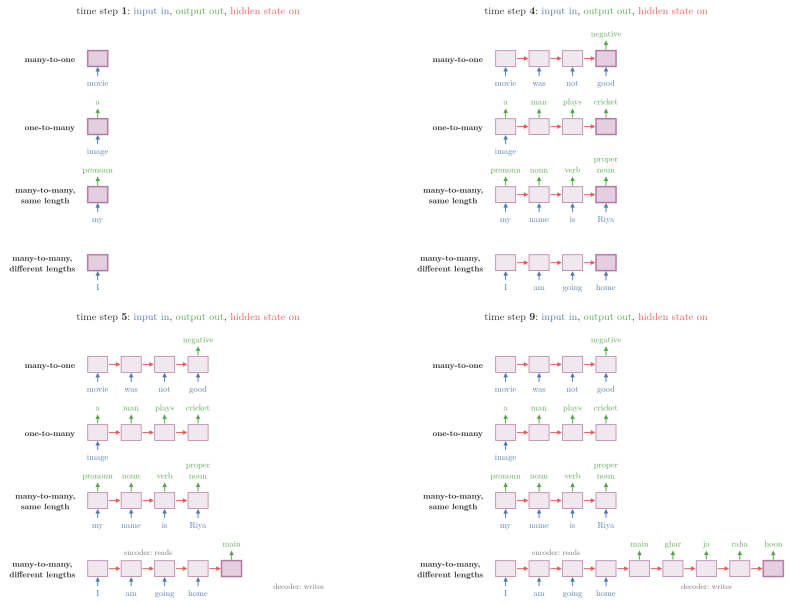


Figure 8: The four types on one time axis, with the examples of this Note. Many-to-one gives its output at the last step only; one-to-many takes its input at the first step only; same-length many-to-many gives an output at every input; with different lengths, the encoder reads all 4 words before the decoder writes 5

- RNN types are named by whether the input and the output are sequences, so the task decides how the recurrent layer is wired.

- Many-to-one reads the whole sequence and predicts once, because the prediction needs a summary of the whole sequence; Keras' default `return_sequences=False`.
- One-to-many turns one input into a sequence, as in **image captioning** (G-918), because the output is a sentence while the input, an image, has no order in time.
- Many-to-many (sequence-to-sequence) predicts at every step when the lengths match, and uses an encoder and a **decoder** (G-564) when they differ, because translation needs the meaning and grammar of the whole sentence before it writes, and a sentence and its translation need not have the same length.
- One-to-one has no time steps, so it is an ordinary ANN or CNN, not an RNN.

9. Sources

Built from

- CampusX, "Types of RNN | Many to Many | One to Many | Many to One RNNs", YouTube, https://www.youtube.com/watch?v=TkOBxzhIySg

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning.* MIT Press. Chapter 10: §10.2 (design patterns for recurrent networks, single output at the end of the sequence), §10.2.4 (vector-to-sequence RNNs, image captioning), §10.4 (encoder-decoder sequence-to-sequence architectures). deeplearningbook.org/contents/rnn.html.
- Keras API documentation: `SimpleRNN` (`return_sequences`), `RepeatVector` and `Dense` (inputs of rank greater than 2) layers, keras.io/api/layers/.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Many-to-one RNN (G-1155)	An RNN that reads a sequence and gives one output at the end.
One-to-many RNN (G-1386)	An RNN that takes one non-sequential input and produces a sequence.
Image captioning (G-918)	Producing a sentence that describes an image.
`RepeatVector` (G-133)	Keras layer that repeats one vector at several time steps.
Many-to-many RNN (G-1154)	An RNN that takes a sequence and produces a sequence; also called sequence-to-sequence.
Sequence-to-sequence (seq2seq) model (G-1771)	A model whose input and output are both sequences, such as a translator from English to Hindi; in RNNs it is the many-to-many type.

Term	Meaning
Same-length many-to-many (G-1721)	Many-to-many with one output per input time step.
Part-of-speech tagging (G-1456)	Labelling every word of a sentence with its part of speech.
Named entity recognition (NER) (G-1300)	Marking which words of a sentence are entities, specific things a program must act on, such as "7 pm" and "airport"; chatbots use it to pick out times and places.
Variable-length many-to-many (G-2072)	A sequence-to-sequence setup (many-to-many) whose output length can differ from its input length.
Machine translation (G-1141)	Translating a sentence from one language into another.
One-to-one (G-1387)	A network with non-sequential input and output: an ordinary ANN or CNN, not an RNN.
Sentiment analysis (G-1769)	Deciding whether a text expresses a positive or negative opinion.
Backpropagation through time (BPTT) (G-246)	Backpropagation for an RNN: the chain rule runs back over the network unfolded in time, adding up the gradient from every time step at which a shared weight is used.
Multi-class classification (G-1266)	Classification where the output is one of more than two classes, such as the next word out of a whole vocabulary; the output layer has one node per class.
Natural language processing (NLP) (G-1305)	The part of ML that works with human language.
`return_sequences` (G-136)	Argument of a Keras recurrent layer: `False` returns only the last hidden state, `True` returns the hidden state at every time step; `True` is needed on every stacked layer but the last.
Encoder (G-682)	The part of a seq2seq model that reads the input sequence and summarises it.
Decoder (G-564)	The part of a seq2seq model that writes the output sequence.
Observation (G-1374)	One record of a dataset: one row of the data table.